# CineMatch

## Movie Recommendation System

## High-Level Design (HLD) Document

*Version 1.0 | April 2026*

Mohd Fuzail , CH22B080

ch22b080@smail.iitm.ac.in

# 1. Executive Summary

CineMatch is a full-stack, production-grade AI application that provides personalized movie recommendations using a content-based filtering approach. The system is built with strict MLOps principles, deployed via Docker Compose as a multi-container architecture, and monitored in near-real-time using Prometheus and Grafana.

The application supports four types of queries: movie title search, genre-based browsing, actor name search, and character name lookup. Each query is served by a FastAPI backend that uses a pre-trained TF-IDF + Cosine Similarity model trained on the TMDB 5000 Movies dataset.

# 2. System Overview

## 2.1 Purpose and Scope

CineMatch addresses the problem of movie discovery overload by providing intelligent, content-aware recommendations. The system is designed for local on-premise deployment with no cloud dependency, making it suitable for privacy-conscious users and resource-constrained environments.

## 2.2 Technology Stack

| Layer | Technology | Rationale |
|---|---|---|
| Frontend | HTML5, CSS3, Vanilla JS, Nginx | Lightweight, no framework overhead, fast load times |
| Backend API | FastAPI (Python 3.11) | Async support, automatic OpenAPI docs, high performance |
| ML Engine | TF-IDF + Cosine Similarity (scikit-learn) | Interpretable, fast inference, no GPU required |
| Experiment Tracking | MLflow 2.12.2 | Industry standard, supports local file-based tracking |
| Pipeline Orchestration | Apache Airflow 2.9.3 | DAG-based ML pipeline automation |
| Monitoring | Prometheus + Grafana | Industry standard for metrics collection and visualization |
| Containerization | Docker + Docker Compose | Reproducible environments, service isolation |
| Data Source | TMDB 5000 Movies + Credits CSV | Well-known benchmark dataset, rich metadata |

# 3. Business Problem & Success Metrics

## 3.1 Problem Statement

Users struggle to discover movies they will enjoy from the vast catalog of available films. CineMatch addresses this by recommending similar movies based on rich content features: genre, cast, director, keywords, and plot overview.

## 3.2 Success Metrics

| Metric | Target | Achieved |
|---|---|---|
| Genre Match Rate | ≥ 70% | ~78% |
| Avg Cosine Similarity | ≥ 0.15 | 0.18 |
| Vocabulary Size | ≥ 5000 | ~6800 |
| Inference Latency | < 200ms | ~120ms |
| Recommendations/query | 10 results | 10 results |
| Poster Availability | ≥ 80% | ~92% |

---

# 4. System Architecture

## 4.1 High-Level Architecture Overview

The system follows a microservices architecture with nine loosely coupled Docker containers communicating over a shared bridge network (cinematch-network). The frontend and backend are strictly separated, connected only via REST API calls proxied through Nginx.

## 4.2 Container Architecture

| Container | Image | Port | Responsibility |
|---|---|---|---|
| cinematch-frontend | nginx:alpine | 5050:80 | Serves HTML/CSS/JS, proxies API calls to backend |
| cinematch-backend | python:3.11-slim | 8001:8000 | FastAPI inference engine, Prometheus metrics exporter |
| mlflow-server | python:3.10-slim | 5001:5000 | Experiment tracking UI, artifact storage |
| airflow-webserver | apache/airflow:2.9.3 | 8080:8080 | DAG management UI, pipeline monitoring |
| airflow-scheduler | apache/airflow:2.9.3 | — | Executes DAG tasks on schedule |
| prometheus | prom/prometheus:latest | 9090:9090 | Scrapes and stores time-series metrics |

| Container | Image | Port | Responsibility |
|---|---|---|---|
| grafana | grafana/grafana:latest | 3001:3000 | Visualizes metrics dashboards in NRT |
| node-exporter | prom/node-exporter:latest | 9100:9100 | Exports host system CPU/memory metrics |
| cadvisor | gcr.io/cadvisor/cadvisor | 8081:8080 | Exports per-container resource metrics |

### 4.3 Request Flow

The user interaction follows this flow:

- User enters a movie name in the browser at http://127.0.0.1:5050
- Nginx receives the request and proxies /api/* to cinematch-backend:8000
- FastAPI backend receives the query and calls ml_pipeline/recommend.py
- recommend.py performs smart query classification (title / genre / actor / character)
- The appropriate search method is executed against the preloaded TF-IDF similarity matrix
- Top 10 results are returned to main.py which enriches them with TMDB metadata
- Results are sorted (searched movie first, then by rating) and returned as JSON
- Frontend renders movie cards with posters, ratings, and overviews

---

## 5. ML Pipeline Design

### 5.1 Pipeline Stages

The ML pipeline is fully automated via an Apache Airflow DAG (cinematch_ml_pipeline) with six sequential tasks:

| Stage | Task | MLflow Logging |
|---|---|---|
| 1. Ingest | Load & validate CSVs, schema checks, quality checks | total_movies, null_rows, total_columns |
| 2. EDA | Analyze raw dataset, rating stats, memory usage | raw_rating_mean, raw_rating_std, memory_mb |
| 3. Preprocess | Parse JSON fields, build weighted tags, save feature store | feature_importance_*, baseline_tag_length_mean |
| 4. Train | TF-IDF vectorization, cosine similarity matrix | vocab_size, vector_rows, vector_cols |
| 5. Evaluate | Genre match rate on 20 random samples | eval_genre_match_rate, eval_avg_similarity |

| Stage | Task | MLflow Logging |
|---|---|---|
| 6. Deploy | Write deployment manifest JSON | genre_match_rate, avg_similarity, deployed_at |

### 5.2 Feature Engineering

The preprocessing pipeline builds a weighted tag string for each movie by concatenating:

- Overview text : 1x weight (contributes ~67% of tag content)
- Genre names : 2x weight (repeated twice for emphasis)
- Keywords : 1x weight
- Top 5 cast members : 2x weight
- Director name : 3x weight (highest emphasis for auteur-based similarity)

All names are cleaned by removing spaces (e.g., 'Tom Hanks' becomes 'tomhanks') to prevent the vectorizer from treating first and last names as separate tokens.

### 5.3 Model Selection Rationale

TF-IDF + Cosine Similarity was chosen over collaborative filtering and neural approaches for the following reasons:

- No cloud requirement : runs entirely on local hardware
- No user interaction data required :content-based from day one
- Fast inference : similarity matrix is precomputed at training time, lookup is O(1)
- Interpretable :recommendations are explainable via tag overlap
- Achieves >80% genre match rate on the TMDB 5000 dataset

### 5.4 Feature Store

Features are versioned separately from model artifacts. Each training run saves a timestamped version directory under ml_pipeline/feature_store/v_YYYYMMDD_HHMMSS/ containing features.csv, baseline_statistics.json, and feature_importance.json. A latest.json pointer tracks the current active version, enabling rollback by simply updating this file.

---

## 6. Monitoring & Observability

### 6.1 Prometheus Instrumentation

The FastAPI backend exposes a /metrics endpoint that Prometheus scrapes every 10 seconds. The following metrics are instrumented:

| Metric Name | Type | Description |
|---|---|---|
| cinematch_requests_total | Counter | Total API requests by method, endpoint, status code |

| Metric Name | Type | Description |
|---|---|---|
| cinematch_request_latency_seconds | Histogram | Request latency distribution per endpoint |
| cinematch_recommendations_total | Counter | Recommendation requests by status (success/not_found/error) |
| cinematch_tmdb_requests_total | Counter | TMDB API calls by status (success/fallback/error) |
| cinematch_tmdb_latency_seconds | Histogram | TMDB API call latency distribution |
| cinematch_active_requests | Gauge | Number of requests currently being processed |
| cinematch_inference_latency_ms | Gauge | Last inference latency in milliseconds (business metric) |
| cinematch_latency_violations_total | Counter | Times inference latency exceeded 200ms SLA threshold |

### 6.2 Grafana Dashboard

The CineMatch Grafana dashboard (auto-provisioned on startup) visualizes ten panels covering: total requests, success rate, inference latency (p50/p95/p99), TMDB API call rate, active requests, latency violations, system CPU usage, and memory usage. Thresholds are configured with green/yellow/red color coding at 150ms and 200ms boundaries.

### 6.3 Business Metrics

Two business-level success metrics are tracked:

- Inference Latency SLA: Maximum 200ms per recommendation request. Violations are counted and alerted.
- Genre Match Rate: Minimum 70% of recommended movies must share at least one genre with the query movie. Evaluated on 20 random samples during each training run.

---

## 7. Key Design Choices & Rationale

| Design Choice | Alternative Considered | Rationale |
|---|---|---|
| TF-IDF + Cosine Similarity | Neural embeddings (BERT) | No GPU required, fast inference, interpretable |
| Content-based filtering | Collaborative filtering | No cold-start problem, no user data needed |
| SQLite for Airflow | PostgreSQL | Simpler setup for local dev; SequentialExecutor compatible |

| Design Choice | Alternative Considered | Rationale |
|---|---|---|
| Local MLflow file store | MLflow with PostgreSQL | Zero external dependencies, simpler Docker setup |
| Nginx reverse proxy | Direct backend access | Strict frontend/backend decoupling, single entry point |
| Weighted tag concatenation | Separate feature vectors | Simple, effective, single vectorizer, easy to tune |
| Director 3x weight | Equal feature weights | Auteur theory — director is strongest predictor of film style |
| Fuzzy title matching | Exact string match | Handles typos and partial titles gracefully |

---

# 8. Security & Reliability

## 8.1 Health & Readiness Checks

The backend exposes two operational endpoints used by Docker health checks and orchestration tools:

- /health — Returns service status and uptime. Used by Docker healthcheck every 30 seconds.
- /ready — Verifies model pkl files exist and TMDB API key is set. Returns 503 if not ready.

## 8.2 Rollback Mechanism

The feature store maintains a latest.json file pointing to the current active model version. Rolling back to a previous model requires only updating this pointer file and restarting the backend container — no retraining required.

## 8.3 Fault Tolerance

- TMDB API failures: Automatic fallback to placeholder poster and N/A rating
- SSL errors: Retry with verify=False as fallback
- MLflow connection failures: Wrapped in try/except, non-critical — recommendations continue
- Model load failures: Fatal error logged, container exits with code 1 for Docker restart
- Missing environment variables: Application refuses to start, logs CRITICAL message

---

# 9. Deployment Architecture

## 9.1 Container Startup Order

Services must start in the following order to avoid dependency failures:

- Step 1: airflow-init — initializes Airflow DB and creates admin user
- Step 2: mlflow-server — must be ready before backend starts
- Step 3: cinematch-backend — depends_on mlflow, loads 177MB similarity.pkl (~3 min startup)
- Step 4: cinematch-frontend — depends_on backend
- Step 5: prometheus, grafana, node-exporter, cadvisor — monitoring stack
- Step 6: airflow-webserver, airflow-scheduler — depends_on airflow-init

### 9.2 Volume Mounts

Critical volumes shared between containers:

- ./ml_pipeline:/app/ml_pipeline — backend accesses trained model pkl files
- ./ml_pipeline:/opt/airflow/ml_pipeline — Airflow tasks access same ML code
- ./mlruns:/mlruns — MLflow server and all pipeline components share experiment data
- ./data:/opt/airflow/data — Airflow accesses raw TMDB CSVs for pipeline runs

| Service | Port | Image | Purpose |
|---|---|---|---|
| cinematch-backend | 8000 | Custom Python 3.11 | FastAPI ML inference API |
| cinematch-frontend | 80 | nginx:alpine | Static file serving |
| cinematch-mlflow | 5001 | ghcr.io/mlflow/mlflow | Experiment tracking UI |
| cinematch-prometheus | 9090 | prom/prometheus | Metrics collection |
| cinematch-grafana | 3001 | grafana/grafana | Metrics visualisation |
| airflow-webserver | 8080 | apache/airflow:2.9.3 | Pipeline UI |
| node-exporter | 9100 | prom/node-exporter | System metrics |

---

## 10. Future Improvements

- DVC pipeline for data and model versioning with CI/CD integration
- Asynchronous TMDB fetching using asyncio.gather (10x speedup for 10 movies)
- Model drift detection by comparing live feature distributions against stored baselines
- User authentication and personalized recommendation history
- A/B testing framework to compare multiple recommendation models
- PostgreSQL backend for Airflow to support parallel task execution

---